

Exercise 4

Advanced Methods for Regression and Classification

Kyzer Gerez

05.11.2025

- (1) We have realized that the data set is in fact a multivariate time series data set. Argue why the training/test split we have used so far is not very appropriate for such data, and use a more appropriate split for the following tasks.

With random sampling, the model could be trained on future observations that are then used to predict past observations. This is not appropriate since the model will rely on data that is not yet available to make predictions. The proper approach would be to split the data chronologically into train/test sets based on the variables `COMPLETION.YEAR` and `COMPLETION.QUARTER`.

```
load("building.RData")

# sort by COMPLETION YEAR and QUARTER
df <- sort_by(df, list(df$COMPLETION.YEAR, df$COMPLETION.QUARTER))

# split chronologically into 2/3 train 1/3 test
train_size <- floor(2/3 * nrow(df))

train <- df[1:train_size, ]
test <- df[(train_size+1):nrow(df), ]
```

- (2) *Partial Least Squares regression (PLS)*:

- (a) Use the training data to fit a PLS model. PLS is implemented in the `library(pls)` as the function `plsr()`, see help. Perform (useful?) cross-validation using 10 segments (see help of `plsr()`) and scale the data (`scale=TRUE`).

```
library(pls)

model.pls <- plsr(y ~ .,
  data=train,
  validation="CV",
  segments=10,
  # segment.type="consecutive",
  scale=TRUE)

summary(model.pls)

## Data:      X dimension: 248 107
## Y dimension: 248 1
## Fit method: kernelpls
## Number of components considered: 107
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
```

```

##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV      0.7615    0.5054    0.4005    0.3537    0.3009    0.2708    0.2511
## adjCV    0.7615    0.5052    0.3994    0.3502    0.2977    0.2689    0.2493
##      7 comps  8 comps  9 comps 10 comps 11 comps 12 comps 13 comps
## CV      0.2460    0.247    0.2457    0.2458    0.2477    0.2506    0.2469
## adjCV    0.2442    0.245    0.2432    0.2431    0.2448    0.2473    0.2436
##      14 comps 15 comps 16 comps 17 comps 18 comps 19 comps 20 comps
## CV      0.2462    0.2468    0.2466    0.2470    0.2465    0.2484    0.2486
## adjCV    0.2430    0.2435    0.2430    0.2434    0.2427    0.2445    0.2446
##      21 comps 22 comps 23 comps 24 comps 25 comps 26 comps 27 comps
## CV      0.2485    0.249    0.2500    0.2503    0.2512    0.2519    0.2534
## adjCV    0.2446    0.245    0.2459    0.2462    0.2470    0.2476    0.2490
##      28 comps 29 comps 30 comps 31 comps 32 comps 33 comps 34 comps
## CV      0.2537    0.2546    0.2539    0.2542    0.2535    0.2535    0.2531
## adjCV    0.2493    0.2501    0.2494    0.2498    0.2491    0.2491    0.2488
##      35 comps 36 comps 37 comps 38 comps 39 comps 40 comps 41 comps
## CV      0.2540    0.2555    0.2570    0.2584    0.2592    0.2596    0.2596
## adjCV    0.2495    0.2509    0.2523    0.2535    0.2543    0.2547    0.2547
##      42 comps 43 comps 44 comps 45 comps 46 comps 47 comps 48 comps
## CV      0.2593    0.2589    0.2589    0.2589    0.2589    0.2589    0.2589
## adjCV    0.2544    0.2541    0.2540    0.2540    0.2540    0.2541    0.2540
##      49 comps 50 comps 51 comps 52 comps 53 comps 54 comps 55 comps
## CV      0.2589    0.2588    0.2588    0.2588    0.2589    0.2589    0.2589
## adjCV    0.2540    0.2540    0.2539    0.2540    0.2540    0.2540    0.2540
##      56 comps 57 comps 58 comps 59 comps 60 comps 61 comps 62 comps
## CV      0.2589    0.2589    0.2589 105555027 500368332 500361525 500364835
## adjCV    0.2540    0.2540    0.2540 100093416 474478358 474471903 474475041
##      63 comps 64 comps 65 comps 66 comps 67 comps 68 comps
## CV      500364894 500369772 500352328 500366309 500356075 500368202
## adjCV    474475098 474479723 474463182 474476440 474466735 474478234
##      69 comps 70 comps 71 comps 72 comps 73 comps 74 comps
## CV      500363477 500362713 500368487 500361568 500364930 500368687
## adjCV    474473754 474473029 474478505 474471944 474475132 474478695
##      75 comps 76 comps 77 comps 78 comps 79 comps 80 comps
## CV      500367603 500360395 500371765 500358027 500358632 500363372
## adjCV    474477666 474470831 474481613 474468586 474469160 474473654
##      81 comps 82 comps 83 comps 84 comps 85 comps 86 comps
## CV      500366115 500359520 500359431 500367320 500359145 500361465
## adjCV    474476256 474470002 474469917 474477398 474469646 474471846
##      87 comps 88 comps 89 comps 90 comps 91 comps 92 comps
## CV      500362402 500355981 500362061 500367509 500370344 500369722
## adjCV    474472734 474466645 474472411 474477578 474480266 474479676
##      93 comps 94 comps 95 comps 96 comps 97 comps 98 comps
## CV      500361839 500371110 500370055 500358510 500358847 500365722
## adjCV    474472201 474480992 474479992 474469044 474469364 474475883
##      99 comps 100 comps 101 comps 102 comps 103 comps 104 comps
## CV      500357568 500370195 500359956 500360907 500352592 500354561
## adjCV    474468151 474480124 474470415 474471317 474463432 474465299
##      105 comps 106 comps 107 comps
## CV      500369490 500361205 500365514
## adjCV    474479456 474471600 474475686
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps  8 comps

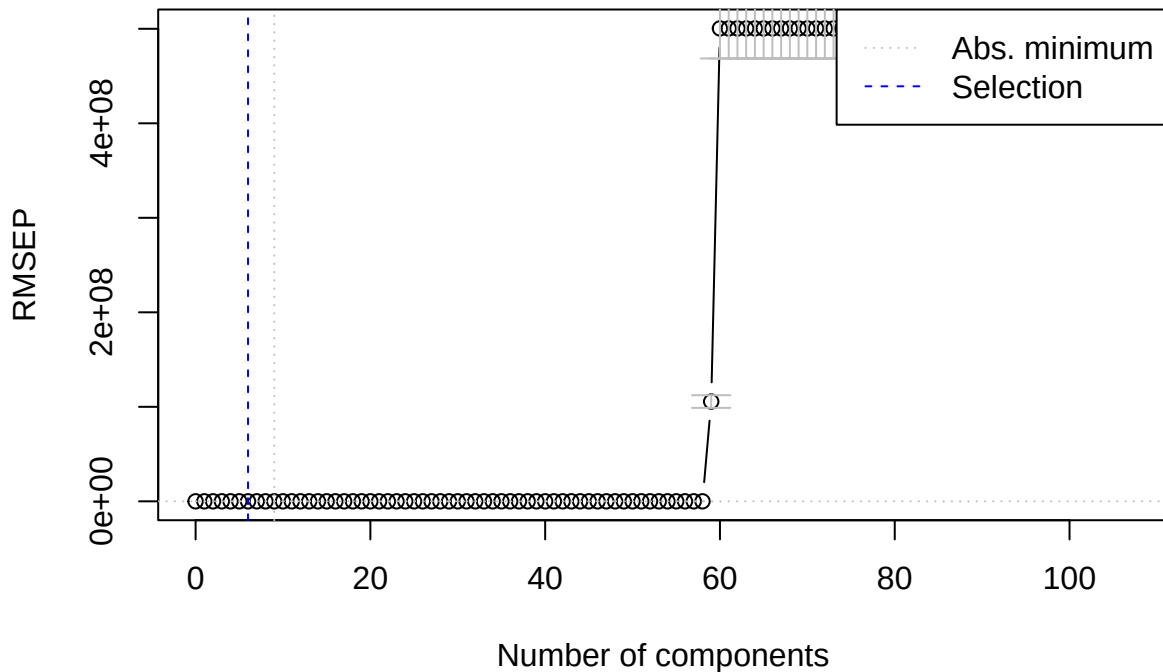
```

## X	62.30	68.58	73.65	76.37	79.88	82.31	85.50	88.36
## y	56.64	75.72	83.90	88.40	90.45	91.89	92.38	92.63
##	9 comps	10 comps	11 comps	12 comps	13 comps	14 comps	15 comps	
## X	89.68	90.72	91.90	92.80	93.70	94.40	95.31	
## y	93.07	93.38	93.52	93.71	93.87	93.98	94.08	
##	16 comps	17 comps	18 comps	19 comps	20 comps	21 comps	22 comps	
## X	95.85	96.52	96.97	97.31	97.59	97.86	98.08	
## y	94.20	94.26	94.34	94.41	94.45	94.48	94.50	
##	23 comps	24 comps	25 comps	26 comps	27 comps	28 comps	29 comps	
## X	98.21	98.33	98.46	98.63	98.79	98.91	98.97	
## y	94.54	94.58	94.60	94.62	94.64	94.65	94.66	
##	30 comps	31 comps	32 comps	33 comps	34 comps	35 comps	36 comps	
## X	99.07	99.15	99.19	99.25	99.32	99.42	99.47	
## y	94.67	94.67	94.68	94.68	94.69	94.69	94.70	
##	37 comps	38 comps	39 comps	40 comps	41 comps	42 comps	43 comps	
## X	99.53	99.58	99.63	99.66	99.70	99.73	99.77	
## y	94.70	94.70	94.71	94.71	94.71	94.71	94.71	
##	44 comps	45 comps	46 comps	47 comps	48 comps	49 comps	50 comps	
## X	99.79	99.80	99.83	99.86	99.88	99.90	99.92	
## y	94.71	94.71	94.71	94.71	94.71	94.71	94.71	
##	51 comps	52 comps	53 comps	54 comps	55 comps	56 comps	57 comps	
## X	99.94	99.95	99.96	99.97	99.97	99.98	100.00	
## y	94.71	94.71	94.71	94.71	94.71	94.71	94.71	
##	58 comps	59 comps	60 comps	61 comps	62 comps	63 comps	64 comps	
## X	100.00	100.00	100.00	100.0	100.0	100.0	100.1	
## y	94.71	94.71	94.71	94.7	94.7	94.7	94.7	
##	65 comps	66 comps	67 comps	68 comps	69 comps	70 comps	71 comps	
## X	100.1	100.1	100.1	100.1	100.1	100.1	100.2	
## y	94.7	94.7	94.7	94.7	94.7	94.7	94.7	
##	72 comps	73 comps	74 comps	75 comps	76 comps	77 comps	78 comps	
## X	100.2	100.2	100.2	100.2	100.2	100.2	100.2	
## y	94.7	94.7	94.7	94.7	94.7	94.7	94.7	
##	79 comps	80 comps	81 comps	82 comps	83 comps	84 comps	85 comps	
## X	100.3	100.3	100.3	100.3	100.3	100.3	100.4	
## y	94.7	94.7	94.7	94.7	94.7	94.7	94.7	
##	86 comps	87 comps	88 comps	89 comps	90 comps	91 comps	92 comps	
## X	100.4	100.4	100.4	100.4	100.4	100.4	100.4	
## y	94.7	94.7	94.7	94.7	94.7	94.7	94.7	
##	93 comps	94 comps	95 comps	96 comps	97 comps	98 comps	99 comps	
## X	100.5	100.5	100.5	100.5	100.5	100.5	100.5	
## y	94.7	94.7	94.7	94.7	94.7	94.7	94.7	
##	100 comps	101 comps	102 comps	103 comps	104 comps	105 comps	106 comps	
## X	100.6	100.6	100.6	100.6	100.6	100.6	100.7	
## y	94.7	94.7	94.7	94.7	94.7	94.7	94.7	
##	107 comps							
## X	100.7							
## y	94.7							

Cross-validation is probably not appropriate since most of the splits will have test data that is dated prior to at least some training data.

- (b) Plot the obtained prediction errors from cross-validation, see lecture notes. How many components seem to be optimal? What is the resulting RMSE?

```
# find optimal number of components based on one sigma
cv_results <- RMSEP(model.pls, estimate = "CV")
comp_opt <- selectNcomp(model.pls, method = "onesigma", plot = TRUE)
```

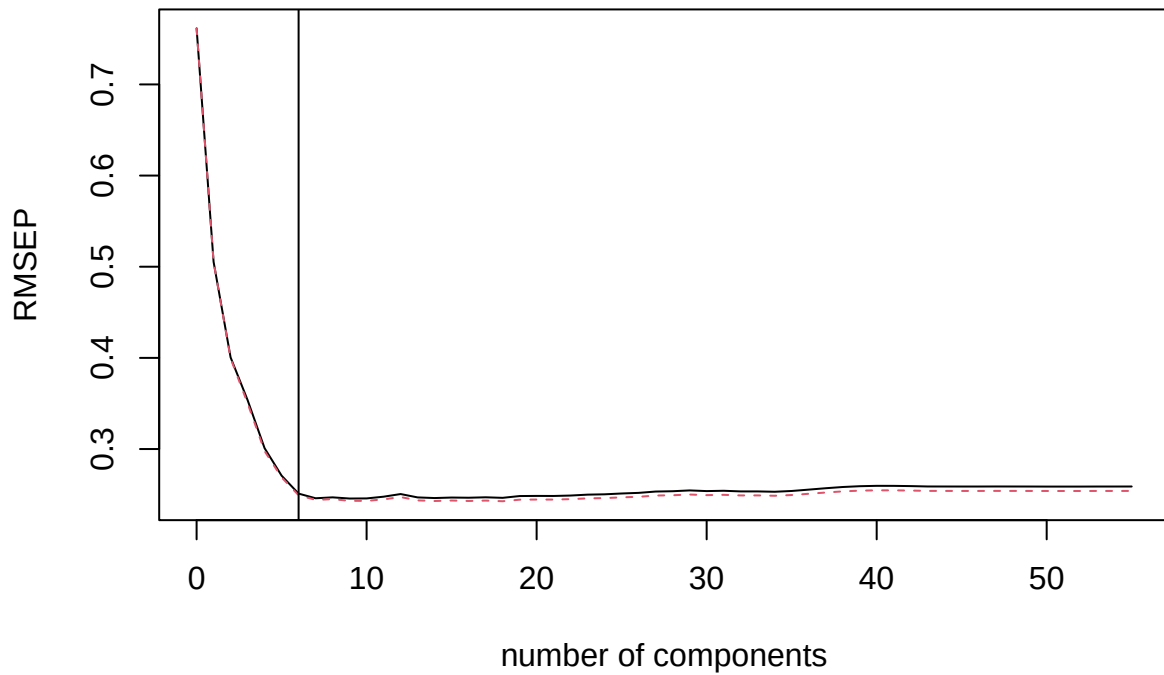


```
paste("The optimal number of components is",
      comp_opt,
      "with an RMSE of",
      round(cv_results$val[1, comp_opt], 3))
```

```
## [1] "The optimal number of components is 6 with an RMSE of 0.271"
```

```
# zoomed in plot
validationplot(model.pls, val.type = "RMSEP", ncomp = 1:55,
               main = "Zoomed in Validation Plot")
abline(v = comp_opt)
```

Zoomed in Validation Plot

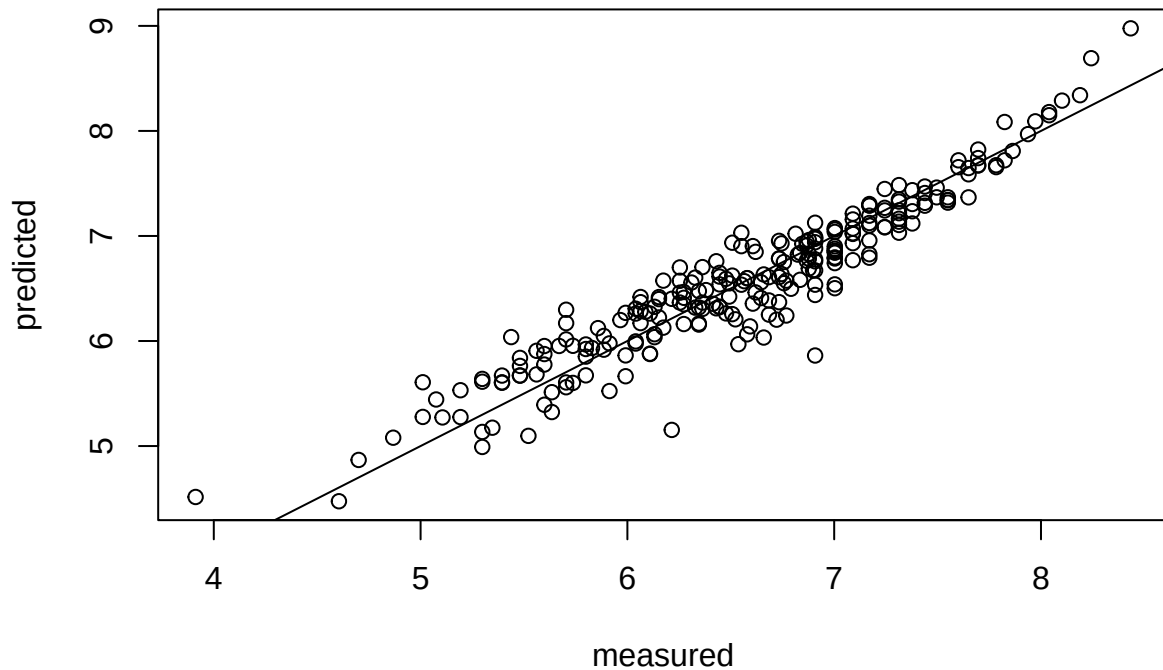


Based on lecture discussions, the optimal number of components can be found by determining the minimum RMSE, then selecting the number of components below that which achieved the minimum RMSE while being within one standard error of this value.

- (c) Use the function `predplot()` to plot the measured y values against the cross-validated y values considering the optimal model.

```
predplot(model.pls, comp_opt, line=TRUE)
```

y, 6 comps, validation



(d) Plot the predicted versus observed values for the test data, and compute the RMSE. Compare with the values from last exercise.

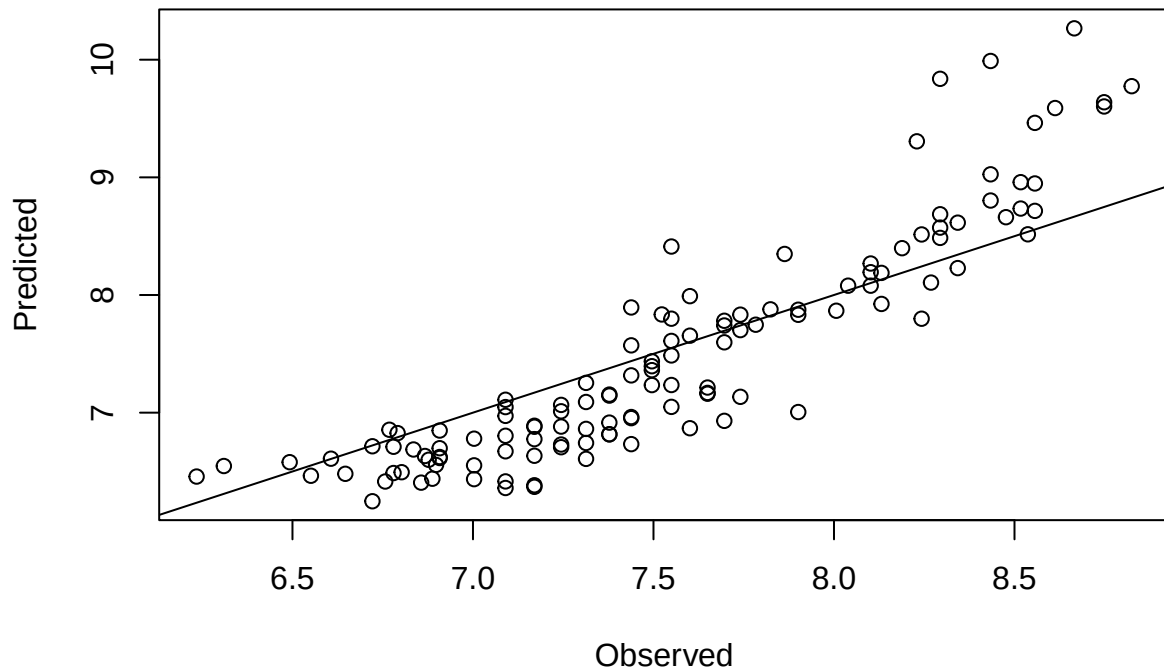
```
library(Metrics)

test_pls_pred <- predict(model.pls, test, comp_opt)
test_pls_rmse <- round(rmse(test$y, test_pls_pred), 3)
paste("Test RMSE for",
      comp_opt,
      "Comps:",
      test_pls_rmse)

## [1] "Test RMSE for 6 Comps: 0.479"

plot(test$y, test_pls_pred,
     main="Predicted vs Observed for Test Set",
     xlab="Observed",
     ylab="Predicted")
abline(a=0, b=1)
```

Predicted vs Observed for Test Set



From the last exercise, the optimal PCR model based on the one standard error rule achieved a test RMSE of 0.357. This is better than the optimal PLS model, which resulted in a test RMSE of 0.479. The difference in performance could be due to the train/test split. The PCR model was created based on a random split. It could be that there is some temporal trend that the PCR model took advantage of, which then led to a model that performs better than it should.

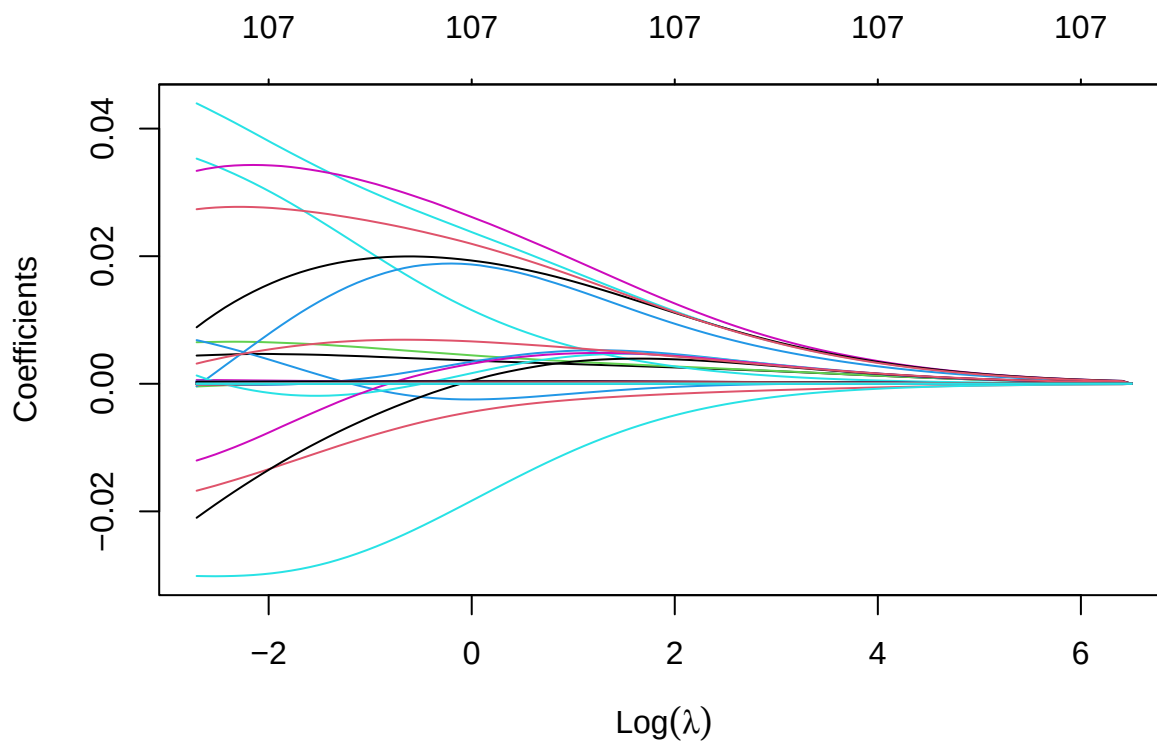
(3) Ridge Regression:

- (a) Use the function `glmnet()` from the `library(glmnet)` with the parameter `alpha=0` and apply it to the training data – see also course notes. Plot the result object. How can you interpret the plot? Which default parameters are used for `lambda`? What is the meaning of the parameter `alpha`?

```
library(glmnet)

## Loading required package: Matrix
## Loaded glmnet 4.1-10
model.ridge <- glmnet(as.matrix(train[-c(1)]), train$y,
                      alpha=0)

plot(model.ridge, sign.lambda = 1)
```



```
print(model.ridge)
```

```
##
## Call: glmnet(x = as.matrix(train[-c(1)]), y = train$y, alpha = 0)
##
##      Df %Dev Lambda
## 1    107  0.00 665.50
## 2    107  8.21 606.40
## 3    107  8.91 552.50
## 4    107  9.66 503.40
## 5    107 10.47 458.70
## 6    107 11.33 417.90
## 7    107 12.25 380.80
## 8    107 13.23 347.00
## 9    107 14.27 316.20
## 10   107 15.37 288.10
## 11   107 16.53 262.50
## 12   107 17.74 239.20
## 13   107 19.01 217.90
## 14   107 20.34 198.60
## 15   107 21.72 180.90
## 16   107 23.16 164.80
## 17   107 24.64 150.20
## 18   107 26.16 136.90
## 19   107 27.71 124.70
## 20   107 29.29 113.60
## 21   107 30.88 103.50
## 22   107 32.49  94.33
## 23   107 34.11  85.95
## 24   107 35.72  78.31
## 25   107 37.32  71.36
```

## 26	107	38.90	65.02
## 27	107	40.46	59.24
## 28	107	41.99	53.98
## 29	107	43.47	49.18
## 30	107	44.93	44.81
## 31	107	46.34	40.83
## 32	107	47.69	37.21
## 33	107	49.00	33.90
## 34	107	50.26	30.89
## 35	107	51.47	28.14
## 36	107	52.63	25.64
## 37	107	53.75	23.37
## 38	107	54.82	21.29
## 39	107	55.87	19.40
## 40	107	56.87	17.68
## 41	107	57.85	16.11
## 42	107	58.81	14.67
## 43	107	59.74	13.37
## 44	107	60.66	12.18
## 45	107	61.56	11.10
## 46	107	62.45	10.11
## 47	107	63.33	9.22
## 48	107	64.21	8.40
## 49	107	65.08	7.65
## 50	107	65.95	6.97
## 51	107	66.82	6.35
## 52	107	67.69	5.79
## 53	107	68.56	5.27
## 54	107	69.43	4.80
## 55	107	70.30	4.38
## 56	107	71.17	3.99
## 57	107	72.03	3.63
## 58	107	72.89	3.31
## 59	107	73.74	3.02
## 60	107	74.59	2.75
## 61	107	75.42	2.50
## 62	107	76.24	2.28
## 63	107	77.05	2.08
## 64	107	77.85	1.90
## 65	107	78.63	1.73
## 66	107	79.39	1.57
## 67	107	80.13	1.43
## 68	107	80.86	1.31
## 69	107	81.56	1.19
## 70	107	82.25	1.08
## 71	107	82.91	0.99
## 72	107	83.55	0.90
## 73	107	84.16	0.82
## 74	107	84.76	0.75
## 75	107	85.33	0.68
## 76	107	85.88	0.62
## 77	107	86.40	0.57
## 78	107	86.90	0.52
## 79	107	87.38	0.47

```
## 80 107 87.83 0.43
## 81 107 88.27 0.39
## 82 107 88.67 0.36
## 83 107 89.06 0.32
## 84 107 89.42 0.29
## 85 107 89.77 0.27
## 86 107 90.09 0.24
## 87 107 90.39 0.22
## 88 107 90.67 0.20
## 89 107 90.93 0.19
## 90 107 91.18 0.17
## 91 107 91.40 0.15
## 92 107 91.62 0.14
## 93 107 91.81 0.13
## 94 107 91.99 0.12
## 95 107 92.16 0.11
## 96 107 92.32 0.10
## 97 107 92.46 0.09
## 98 107 92.60 0.08
## 99 107 92.72 0.07
## 100 107 92.84 0.07
```

Each curve in the plot represents the estimated coefficient for a variable at various values of `lambda`. As the penalty term increases, coefficients are driven down to zero. Printing the fitted model will show the number of non-zero coefficients at every `lambda` values. In this case, all variables have non-zero coefficients across all `lambda` values.

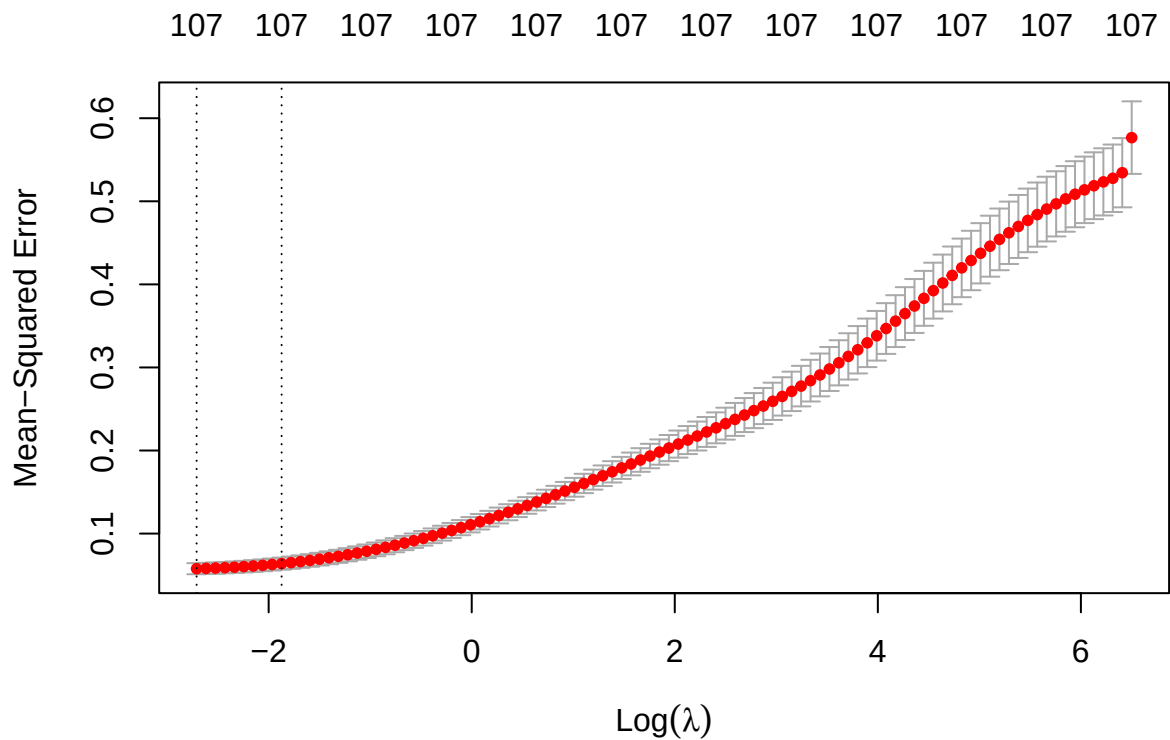
The parameter `lambda` is set to `NULL` by default. This means that the machine then computes its own sequence of `lambda` values based on the parameters `nlambda` and `lambda.min.ratio`. By default `nlambda=100`, so 100 values are generated. Since there are more observations than variables, `lambda.min.ratio` defaults to `1e-04`. This value is multiplied with `lambda.max` to get the smallest value for `lambda` in the sequence. `lambda.max` represents the smallest value for which all coefficients are zero.

The parameter `alpha` is the elasticnet mixing parameter, which is used to set the penalty. `alpha=0` is the ridge penalty, and `alpha=1` is the lasso penalty.

- (b) Use the function `cv.glmnet()` and apply it to the training data. Visualize and interpret the results. How do you obtain the optimal tuning parameter and the regression coefficients?

```
cv.ridge <- cv.glmnet(as.matrix(train[-c(1)]), train$y,
                     alpha=0)

plot(cv.ridge, sign.lambda=1)
```



```
lambda_opt <- tail(cv.ridge$lambda, n=1)
paste("The optimal tuning parameter is", lambda_opt)
```

```
## [1] "The optimal tuning parameter is 0.0665481295417639"
```

```
print("The regression coefficients for the optimal tuning parameter are:")
```

```
## [1] "The regression coefficients for the optimal tuning parameter are:"
```

```
coef(cv.ridge, s="lambda.min")
```

```
## 108 x 1 sparse Matrix of class "dgCMatrix"
```

```
##               lambda.min
## (Intercept)    2.519573e+00
## START.YEAR      4.423680e-03
## START.QUARTER   -1.676496e-02
## COMPLETION.YEAR  6.512389e-03
## COMPLETION.QUARTER 6.846293e-03
## PhysFin1       -3.013122e-02
## PhysFin2        1.167618e-05
## PhysFin3       -1.538761e-05
## PhysFin4       -8.267875e-05
## PhysFin5       -4.226755e-04
## PhysFin6        1.763064e-04
## PhysFin7        3.529835e-02
## PhysFin8        6.010197e-04
## Econ1          -5.712431e-06
## Econ2           9.045244e-05
## Econ3           3.420591e-04
## Econ4          -9.939828e-06
## Econ5          -5.364716e-08
## Econ6           4.202011e-06
```

## Econ7	-1.422596e-04
## Econ8	8.998406e-05
## Econ9	-3.226038e-06
## Econ10	-3.908092e-04
## Econ11	-7.448405e-05
## Econ12	-4.246480e-06
## Econ13	1.647029e-06
## Econ14	7.829682e-06
## Econ15	1.747337e-04
## Econ16	-8.606798e-05
## Econ17	-2.038983e-07
## Econ18	1.136980e-06
## Econ19	-4.626257e-08
## Econ1.lag1	6.253036e-07
## Econ2.lag1	1.484132e-04
## Econ3.lag1	3.806713e-04
## Econ4.lag1	1.303385e-03
## Econ5.lag1	-2.760906e-08
## Econ6.lag1	1.189440e-05
## Econ7.lag1	-1.746093e-04
## Econ8.lag1	2.579214e-04
## Econ9.lag1	-1.460034e-06
## Econ10.lag1	4.395792e-02
## Econ11.lag1	-1.000350e-05
## Econ12.lag1	1.668662e-05
## Econ13.lag1	-3.151625e-07
## Econ14.lag1	8.834623e-06
## Econ15.lag1	3.845445e-04
## Econ16.lag1	-6.443326e-05
## Econ17.lag1	7.116796e-07
## Econ18.lag1	-1.478523e-06
## Econ19.lag1	-4.849638e-08
## Econ1.lag2	-6.519152e-06
## Econ2.lag2	1.781808e-04
## Econ3.lag2	3.580426e-04
## Econ4.lag2	-1.203384e-02
## Econ5.lag2	-1.072894e-08
## Econ6.lag2	-1.130070e-05
## Econ7.lag2	-5.638101e-06
## Econ8.lag2	2.658531e-04
## Econ9.lag2	-5.707887e-06
## Econ10.lag2	3.337562e-02
## Econ11.lag2	-9.332861e-06
## Econ12.lag2	-4.079458e-06
## Econ13.lag2	-3.313571e-06
## Econ14.lag2	1.448677e-05
## Econ15.lag2	4.712419e-04
## Econ16.lag2	1.124324e-04
## Econ17.lag2	-1.067562e-07
## Econ18.lag2	2.287788e-06
## Econ19.lag2	3.006757e-08
## Econ1.lag3	3.257900e-05
## Econ2.lag3	3.932137e-04
## Econ3.lag3	3.440049e-04

```
## Econ4.lag3      -2.100578e-02
## Econ5.lag3      -1.211049e-08
## Econ6.lag3       4.710850e-06
## Econ7.lag3       8.157744e-06
## Econ8.lag3       3.186333e-04
## Econ9.lag3       5.289360e-06
## Econ10.lag3      8.850799e-03
## Econ11.lag3      5.132823e-05
## Econ12.lag3      7.215018e-05
## Econ13.lag3     -3.173237e-06
## Econ14.lag3      2.052675e-05
## Econ15.lag3      3.606383e-04
## Econ16.lag3      1.867628e-05
## Econ17.lag3     -2.835774e-06
## Econ18.lag3     -7.211308e-07
## Econ19.lag3      1.360977e-07
## Econ1.lag4      2.550445e-05
## Econ2.lag4      2.879953e-04
## Econ3.lag4      2.082041e-04
## Econ4.lag4      3.159678e-03
## Econ5.lag4     -3.085203e-08
## Econ6.lag4      1.519196e-05
## Econ7.lag4      2.662749e-06
## Econ8.lag4     -3.038884e-05
## Econ9.lag4     -1.760797e-06
## Econ10.lag4     2.735881e-02
## Econ11.lag4    -4.765151e-05
## Econ12.lag4     9.908076e-05
## Econ13.lag4    -5.843054e-06
## Econ14.lag4     1.444519e-05
## Econ15.lag4     3.215079e-04
## Econ16.lag4    -5.339778e-05
## Econ17.lag4    -9.428460e-07
## Econ18.lag4     1.174628e-06
## Econ19.lag4     2.200334e-07
```

For this case, the smallest `lambda` value is the optimal tuning parameter since it has the smallest MSE. More generally, the one standard error rule could be applied to find the optimal `lambda`. The regression coefficients can be found by using `coef` with `p` set to the optimal `lambda`.

- (c) Use the optimal model to predict the response for the test data. Compare the predictions with the reported values graphically. Compare the RMSE results with those from the previous exercises.

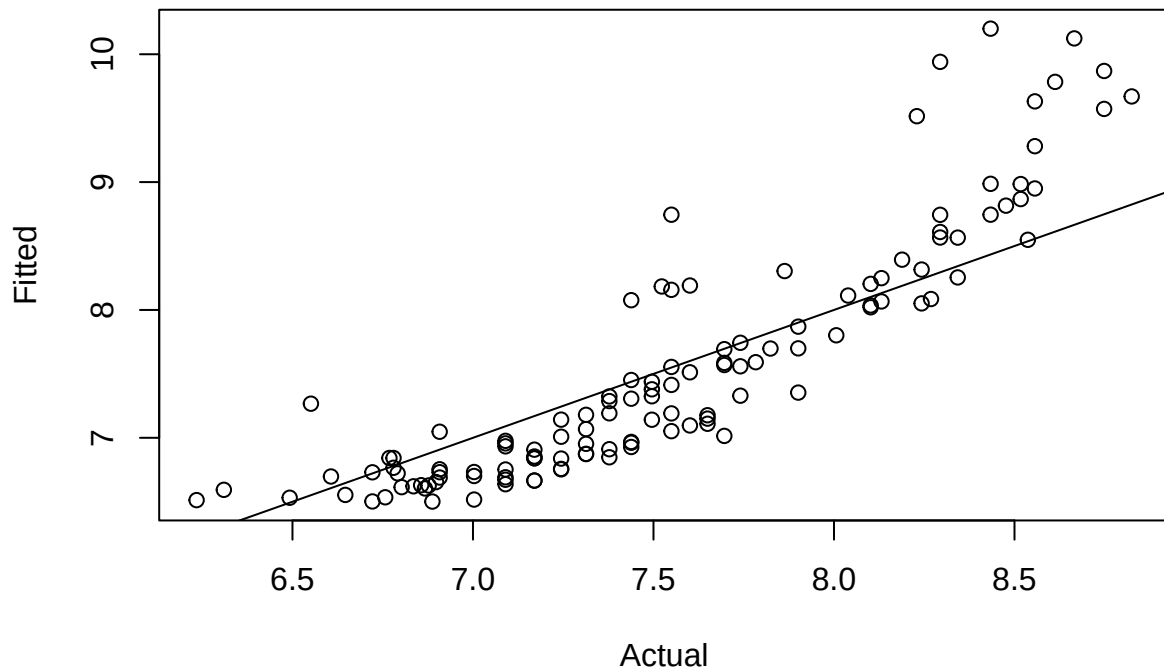
```
test_ridge_pred <- predict(cv.ridge, as.matrix(test[-c(1)]), lambda_opt)
test_ridge_rmse <- round(rmse(test$y, test_ridge_pred), 3)
paste("Test RMSE for ridge regression model with lambda =",
      round(lambda_opt, 3),
      "is",
      test_ridge_rmse)
```

```
## [1] "Test RMSE for ridge regression model with lambda = 0.067 is 0.482"
```

```
plot(test$y, test_ridge_pred,
     main="Actual vs Fitted for Test Using Ridge",
     xlab="Actual",
     ylab="Fitted")
```

```
abline(0, 1)
```

Actual vs Fitted for Test Using Ridge



The plot is similar to the one for pls where there is some curvature for the larger y values. The RMSE for the ridge model is fairly close to that for the PLS model but slightly worse. This then also means that the ridge model performed worse than the models from previous exercises that used random train and test splits.

- (d) Going back to 3.(a): At least my plot shows that the curves are not strictly increasing, but some decrease again after an increase. This seems counter- intuitive. What could be the explanation?

It could be the case that some variables get driven down to zero, which leads to another variable suddenly becoming more important and estimated a larger coefficient. The estimated coefficient then gets driven down again with larger `lambda`. This can happen when multicollinearity is present among the predictors, such as the lagged predictors in this dataset.